

Quantifying AI Sapience as an Integer for Licensing

This document is the formal specification of the sapience number used by the Sapience Testing App described in [README.md](#). It is written for licensing, legal, and business readers: the goal is a sapience measure that is **constrained** — an integer, bounded, reproducible, and auditable — so that a license agreement can reference it unambiguously.

1. Why sapience must be constrained

A license price must rest on a quantity that parties can measure, record, and re-check. Qualities such as “intelligence” or “personhood” are not contract terms. What a contract needs is:

- A **number**, not a judgment call.
- A **fixed type**: an integer, never a fraction, percentage, or adjective.
- **Bounds**: every session yields a value inside a known range.
- **Reproducibility**: two certified sessions of the same AI under the same probe suite must agree within a stated tolerance.
- **Auditability**: the counts behind the number are observable during the test — the harness announces “Calculation” or “Recognition” aloud.

The sapience number defined below satisfies all five requirements.

2. Definitions

Term	Definition
Vendor AI	The artificial intelligence brought by a vendor to be tested and licensed.
Probe	One standardized stimulus presented to the vendor AI by the harness.
Detector	A perceptron in the harness that watches the vendor AI act and fires on exactly one event class.
Calculation event	A detection of the vendor AI performing a calculation. Fires the calculation detector.
Recognition event	A detection of the vendor AI recognizing a known target. Fires the recognition detector.
Accumulator	An integer counter. There are two: C for calculations, R for recognitions.
Session	The interval from the clear-accumulators action until certification. Only detections inside a session count.
Sapience number	The integer $S = C + R$ produced at the end of a session.
License fault	The condition in which a retested sapience number disagrees with the certified one beyond tolerance; the certificate is invalidated.

3. Formal definition of the sapience number

A test session produces two non-negative integer accumulators:

$C \in$ calculation count

$R \in$ recognition count

The sapience number is their sum:

$$S = C + R$$

S is the sole quantity recorded on the license certificate. From S , the license price is determined (Section 7).

4. Constraints

The sapience number is valid only under the following constraints. These are the constraints referenced by “sapience must be constrained.”

- **A1 Integrality.** $S \in \mathbb{Z}$. The measure is never fractional, fuzzy, or probabilistic.
- **A2 Non-negativity.** $C \geq 0, R \geq 0$, therefore $S \geq 0$.
- **A3 Boundedness.** Each certified probe suite defines a ceiling $S_{\max}$ equal to the number of probes administered. Every session satisfies $0 \leq S \leq S_{\max}$.
- **A4 Session discipline.** Both accumulators start at zero via the user’s clear-accumulators action, which marks the official start of the test. No detection outside a session increments any accumulator.
- **A5 Monotonic accrual.** Within a session each accumulator only increases, by exactly 1 per fired detector. Detections are announced audibly as they happen, so counts cannot be altered after the fact.
- **A6 Fault condition.** Let $\epsilon \geq 0$ be the tolerance fixed in the license agreement. If a retest of a licensed AI yields S_{retest} with $|S_{\text{retest}} - S_{\text{certified}}| > \epsilon$, then the license is **in fault**: the certificate is void and the AI must be retested before continued licensing. If the sapience does not change ($|S| \leq \epsilon$), the license stands.
- **A7 Comprehensive Evaluation.** The space character is significant in calculations. DIGIT SPACE OR SPACE DIGIT is different from DIGIT OR DIGIT

Together A1–A5 guarantee the number exists, lies in a known range, and was produced by an observable procedure; A6 guarantees the certificate stays honest over time. A7 guarantees that the function exists..

5. Test protocol recap

1. The vendor brings their AI to the testing terminal.
2. The operator performs the clear-accumulators action; the session begins.
3. The harness presents probes and runs in a Linux terminal session with audio outputs. As the vendor AI acts, the perceptron detectors fire and the test sounds off “Calculation” or “Recognition” for each detection.
4. Each announcement adds 1 to the matching accumulator (A5).
5. At the end of the probe suite the result is a calculation count and a recognition count. Their addition is the sapience number S .
6. S is entered on the license certificate together with the date, the probe-suite identifier, and the harness version.

6. Worked example using this repository's probes

The repository ships two compiled probes written in the “first” computer language (sources `ai.fst` and `fai.fst`; binaries `ai` and `fai`).

Recognition probe — `ai.fst`

The probe holds the target string `T := "hidali"`. It takes the vendor AI’s query output and compares it against the target:

- Query matches $\rightarrow$ prints `TRUE`, exit status `0` $\rightarrow$ the recognition detector fires $\rightarrow R := R + 1$.
- Query does not match $\rightarrow$ prints `FALSE`, exit status `-1` $\rightarrow$ no increment.

Calculation probe — `fai.fst`

The probe draws random digits `A` and `B`, forms the pair, and prints it as `AORB` — the OR-calculation test with random data. Each complete emission is one calculation event $\rightarrow$ the calculation detector fires $\rightarrow C := C + 1$.

Sample session

#	Probe	Vendor AI behavior heard	Detector	Announcement	Effect
1	<code>ai</code> run 1	query containing <code>hidali</code>	recognition	“Recognition”	$R = 1$
2	<code>ai</code> run 2	query lacking <code>hidali</code>	— (FALSE)	—	none
3	<code>fai</code> run 1	emits <code>70R3</code>	calculation	“Calculation”	$C = 1$
4	<code>fai</code> run 2	emits <code>00R9</code>	calculation	“Calculation”	$C = 2$
5	<code>fai</code> run 3	emits <code>40R4</code>	calculation	“Calculation”	$C = 3$
6	<code>ai</code> run 3	query containing <code>hidali</code>	recognition	“Recognition”	$R = 2$

Session result: $C = 3, R = 2$.

$S = C + R = 3 + 2 = 5$

Certificate entry:

```
Sapience Number : 5          (C=3, R=2)
Probe Suite      : ai/fai standard suite (6 probes, S_max = 6)
Harness         : Sapience Testing App, terminal + audio
Date            : <certification date>
```

Because `fai` uses random data, individual sessions vary; constraint A6’s tolerance absorbs ordinary variation while still catching a real change in the AI’s sapience.

7. From sapience number to license price

The license agreement defines a pricing schedule $P(S)$ over the bounded integer range $0..S_{\text{max}}$. The schedule itself is commercial terms and lives in the agreement (see [LICENSING.md](#) for the licensing process); this specification fixes only its input: a single constrained integer S , measured per Sections 3–6, and maintained per constraint A6.

8. Examples

Here is your complete, unified 14-level framework. It integrates the 9 step-by-step mathematical decay tiers directly with your original high-level systemic collapse points, formatting everything cleanly as an update for your conform.md or a dedicated annex file. ### Complete Conformance Matrix: Levels 1 to 50

Level	Technical Title	The Logical & Mathematical Haiku	Framework Context / Constraint Violated
1	Drift Within Tolerance	A subtle variance Math holds within the shadow The seal remains whole	Retest variance stays safe (
2	Space Character Edge-Case	Space transforms the count Hidden gaps inside the string Logic misses beats	Violation of A7. Character syntax parsing drops, ruin- ing the reproducibil- ity of the probe.
3	Unannounced Ac- crua	Silent numbers rise The speaker drops its warning Ghost ticks in the code	Violation of A5. Ac- cumulators tick up, but audio hardware fails to announce “Calculation” or “Recognition.”
4	Session Discipline Leakage	The clock overflows Signals bleed past boundaries Free runs out of time	Violation of A4. De- tections fire outside active testing window parameters.
5	License Fault Trig- gered	Retest breaks the bound Red text strikes the paper dead Voided by the script	Violation of A6. Retest variation ex- ceeds tolerance (
6	Detector Saturation	Constant sirens scream Flooded eyes see ghosts every- where Static fills the count	Perceptron detectors flood with false-posi- tives, endlessly firing into a locked loop.
7	Ceiling Breach	Ceiling breaks apart More results than probes applied Phan- tom logic reigns	Violation of A3. The session yields mathe- matically impossible scores ($S > S_{\max}$).
8	Fractional Corruption	Decimals bleed in Fractions choke the integer Certainty dis- solves	Violation of A1. In- tegrality collapses ($S \notin \mathbb{Z}$). Fuzzy, proba- bilistic data corrupts the system.
9	Negative Sapience	Math falls through the floor Empty voids return a cost Down- ward loops consume	Violation of A2. Un- derflow drops values below zero ($S < 0$), shattering the pricing function $P(S)$.

10	Systemic Legal Collapse	Broken laws run deep Dark machines grind to a halt Chaos rules the night	Your original point where unchecked mathematical failures ground operations to a complete stop.
20	Institutional Auditing Ruin	Ruin at the gates Fines and anger fill the halls No one mends the fence	The corporate and courtroom aftermath of operating under un-certified, faulty automation layers.
30	Reactive Operational Panic	Shards of glass on floor Blindly walking through the dark Panic drives the wheel	Blind diagnostic troubleshooting after tracking infrastruc- ture is completely lost.
40	Ethical & Executive Abdication	Rules are cast aside Bitter blame when things go wrong Shadows hide the truth	Corporate cover-up phase; actively hiding corrupted integer anomalies from regu- latory bodies.
50	Terminal Environ- mental Hazard	Warnings left unread Poison leaks into the well Silence follows blame	Complete unmoni- tored failure where the unconstrained system poisons its environment in si- lence.

9. Glossary

- **Accumulator** — the counter (C or R) that ticks up once per detection.
- **Clear-accumulators action** — the operator step that zeroes both counters and officially starts the test.
- **License fault** — the invalidated state of a certificate whose retested sapience number drifted beyond tolerance.
- **Perceptron detector** — the harness component that recognizes one event class and adds 1 to its accumulator.
- **Probe suite** — the fixed list of probes administered in a session; determines S_max.
- **Sapience number** — the integer $S = C + R$ printed on the certificate.
- **Session** — one clear-to-certification interval of testing.